Production React 19 Exercise Report

Semantic components, derived state and abortable asynchronous workflows.

Student	Submission information
Full name	Dang Hoang Long
Student ID	24520995
Submission date	02 November 2026
Scope	Exercises 1-3 + Bonus Situationship FSM
Repository folder	Lab_03/Exercise

Objective and source

Implement the in-class exercise contracts in the supplied 35-slide Lab 03 presentation: profile card (slides 13-14), scientist directory (18-19), PUBG Vietnam telemetry (22-23), and the creative bonus (31-32). The separate Homework 1-3 section is outside this report.

Delivered work

A React 19/Vite application includes all four exercise views, engineering rules, task decomposition, an executable failure audit, automated tests, accessibility evidence and screenshots. Each exercise can be opened independently from the app navigation.

Source repository:
github.com/mikeywrld/24520995_DangHoangLong_Web

Target folder after push:
Open Lab_03/Exercise after publication

Publication status: Ten local commits are prepared. GitHub write requests returned 403 (Resource not accessible by integration), and direct Git has no authenticated push credential. The source has not been published during this session. A commit-preserving patch accompanies the submission.

01 / EXERCISE 1

Semantic Developer Profile

The profile composes DeveloperCard and SkillBadge. A labelled article contains a header, a description list and a list of skills. Object.entries maps metrics through keyed Fragments, so no layout wrappers or duplicate headings are required.

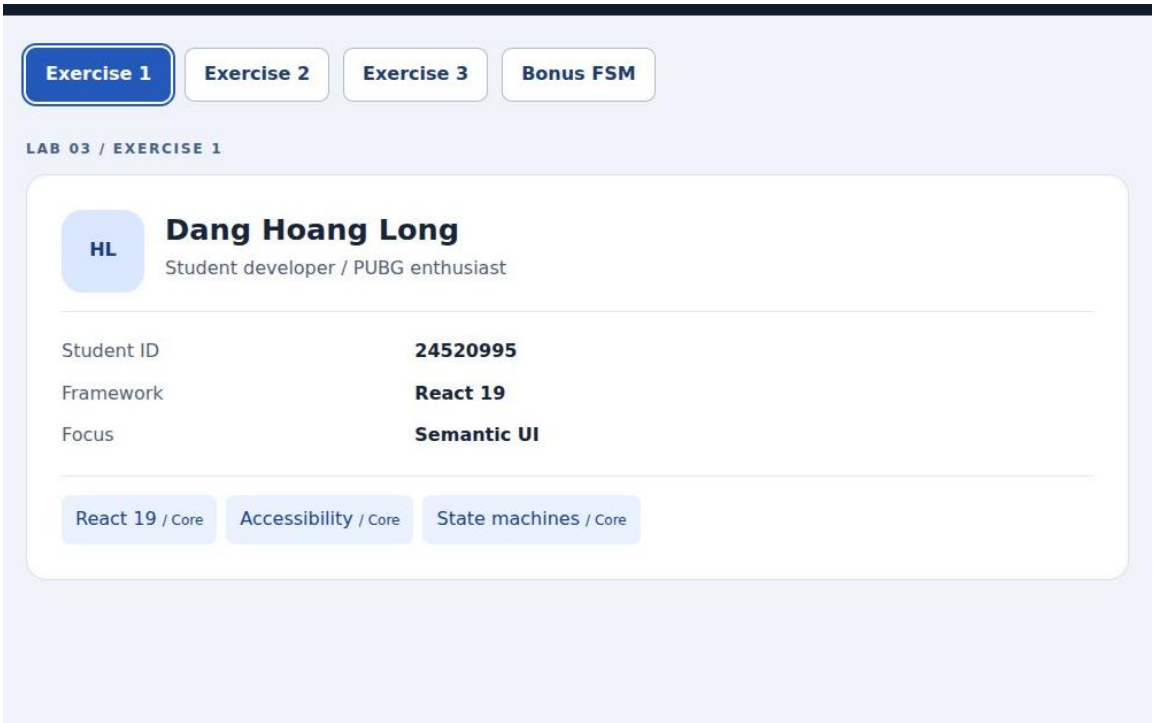


Figure 1. Actual production build. The browser test blocks the HTTPS avatar, demonstrating the accessible HL initials fallback.

Contract	Implementation
Zero div tags	DeveloperCard.jsx uses article, header, hgroup, dl, dt, dd, ul, li and Fragment.
React 19 baseline	Function components, default parameters, createRoot and StrictMode.
Media safety	HTTPS-only source, explicit 72 x 72 dimensions, failure fallback.
CSS isolation	DeveloperCard.module.css with scoped classes and --accent custom property.
Identity and accessibility	useId links the article and heading. Skill IDs and metric names remain stable.

Verification

The browser test confirms the card mounts, contains zero descendant div elements and retains a named image alternative after an avatar failure. Axe reports zero automated WCAG violations. The 375px screenshot on page 7 shows the header and skills wrapping cleanly.

02 / EXERCISE 2

Interactive Scientist Directory

The directory stores only independent inputs: people, query, minimum IQ and sort order. selectPeople calculates the visible list during render. No filtering or sorting effect exists. Search trims whitespace and compares lowercase text. Numeric input uses valueAsNumber with a zero fallback.

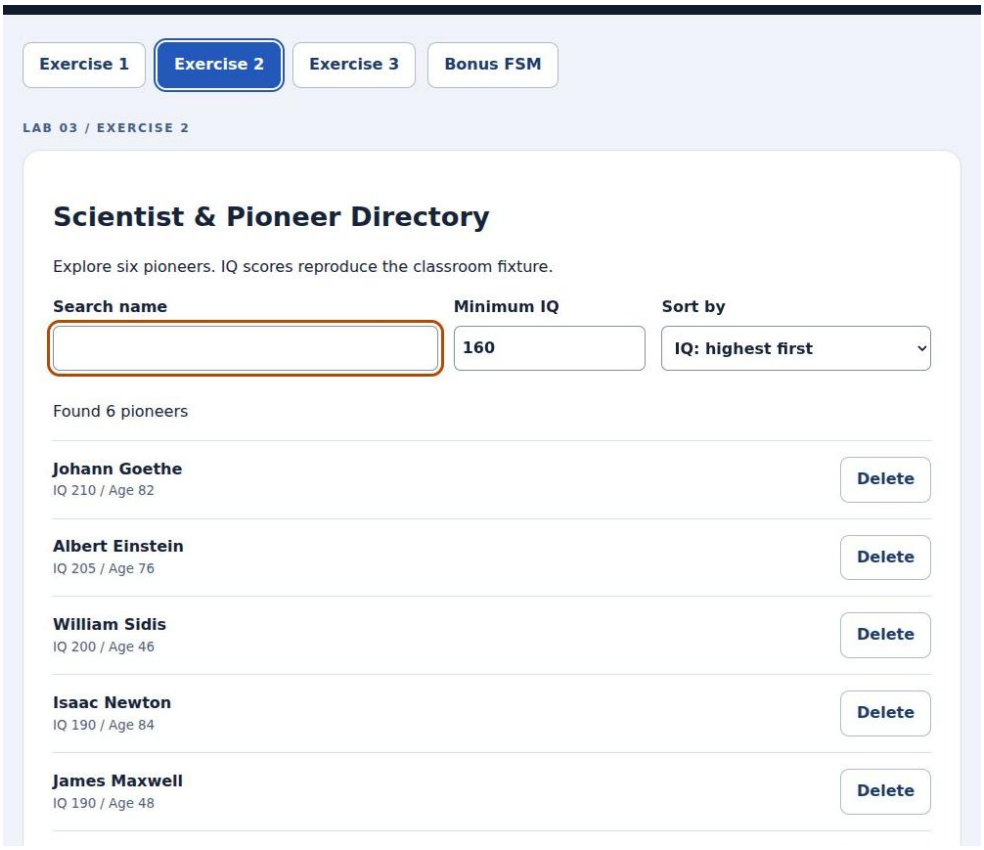


Figure 2. Directory after reset, with six pioneers restored and filters returned to their defaults.

Immutable updates and stable rows

The source fixture uses stable IDs sp-1 through sp-6. Filtering and toSorted return new arrays. Delete applies prev.filter, while Reset restores a copy of the initial fixture and resets the controls. A polite live output announces the result count. Delete and Reset move focus to the search input, preventing focus loss when a row disappears.

Observed test outcomes

Searching for " EINSTEIN " returns one result. A minimum IQ of 211 returns zero results. Sorting by age places William Sidis first. Deleting him leaves five results and restores search focus. Reset returns all six records. These results are recorded in evidence/browser-results.txt.

The IQ scores reproduce the slide fixture as teaching data. They are not presented as verified historical measurements.

03 / EXERCISE 3

PUBG Vietnam Telemetry

A local asynchronous service simulates CES, TE and GE squad data with a 900ms delay. Random mode rejects samples below 0.35. Deterministic success/failure controls make retry and error states reproducible. This is simulated telemetry, with no official KRAFTON API connection.

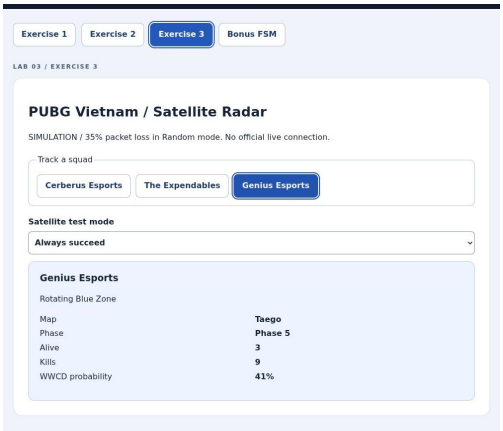


Figure 3. After rapid squad switching, Genius Esports remains selected and its payload wins.

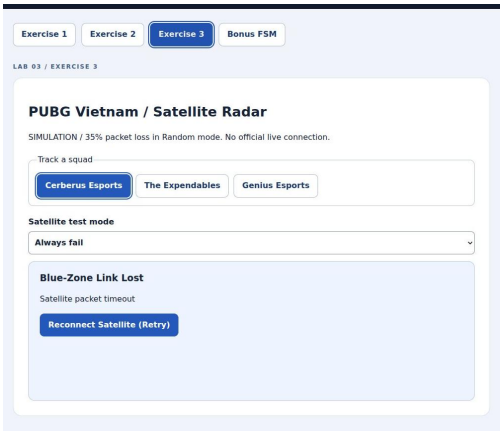


Figure 4. Forced packet loss produces an accessible alert and a Reconnect Satellite (Retry) action.

Lifecycle and state integrity

The view uses IDLE, LOADING, SUCCESS and ERROR. Every effect creates an AbortController and aborts it on squad/mode changes, retry or unmount. AbortError is ignored. A signal guard also prevents stale results from updating the view. Retry increments a counter to rerun the effect.

Loading displays a 220px pulsing skeleton within a fixed 280px radar frame shared by all states. The fixed frame avoids panel-height jumps. Whole-application zero CLS has not been inferred from this sizing rule. Because the service uses local timers, its cancellation does not create canceled HTTP rows in DevTools Network.

Situationship State Machine

The implementation follows the prose phase names on slide 31. Its reference code uses different names; the chosen mapping is FRIENDS = STRANGERS, OFFICIAL_LOVE = COMMITTED, and GHOSTED_EXIT = SELF_RESPECT_EXIT.

Current phase	Event / next phase
STRANGERS	Start dating -> DATING_NO_LOVE
DATING_NO_LOVE	Send message -> wait in same phase
DATING_NO_LOVE	Confirm mutual reply -> COMMITTED
DATING_NO_LOVE	Matching message timeout -> SELF_RESPECT_EXIT
Any phase	Stop -> SELF_RESPECT_EXIT; Reset -> STRANGERS

Immutable effort signals store actor, steps and multiplier. Scores are derived by reduction during render. With 100 personal steps and one rival step, the scores are 100 x 0.1 = 10 and 1 x 50 = 50. The dating phase announces a red flag when personal impact is lower.

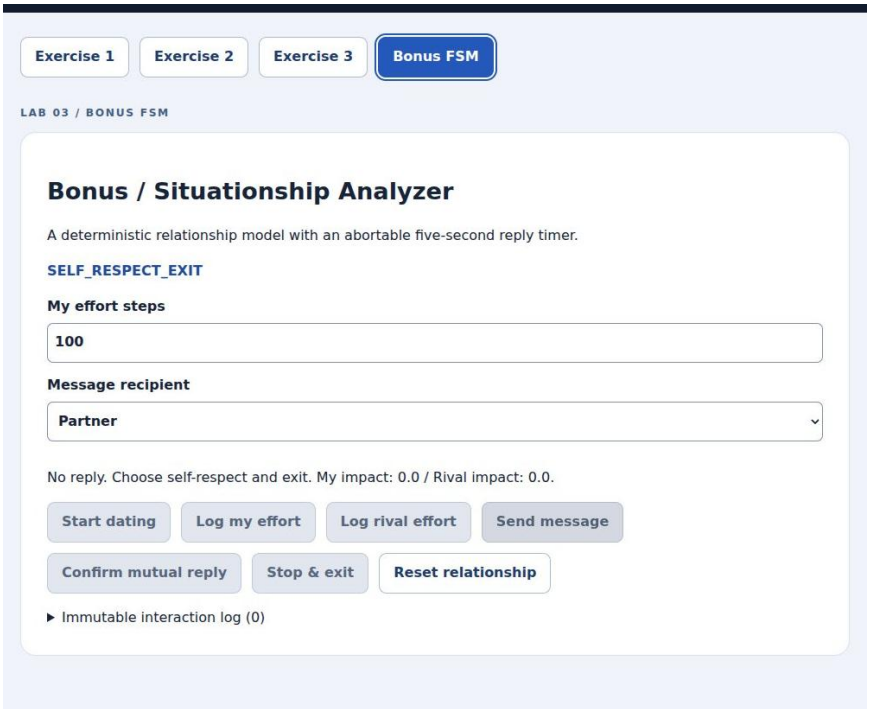


Figure 5. Actual unanswered-message outcome after the full five-second production delay.

Sending starts the reply wait. Reply, recipient change, stop, reset and navigation unmount abort pending work. TIMEOUT must match the current message ID. Tests confirm that a mutual commitment survives beyond five seconds and that unanswered messages exit. Cleanup releases timer/listener references; JavaScript garbage collection remains browser controlled.

Automated Results and Limits

Check	Observed result
Production build	Vite 8.3.5 build succeeded with React 19.3.1.
Unit tests	11 passed, zero failed; see test-results.txt.
Failure audit	Three faulty candidates reproduced, then repaired behavior asserted.
Browser scenarios	Search/filter/sort/delete/reset, errors/retry, rapid switching and full five-second FSM behavior passed.
375px layout	No document horizontal overflow on all four views.
Form labels	All input/select controls have associated labels on all four views.
axe scans	Zero automated violations on each of the four exercise views.
Lighthouse	Accessibility: 100/100 for the initial Exercise 1 route only.
Console	Zero JavaScript runtime errors or warnings during scripted scenarios.

Reproduction

npm ci
npm run build
npm test
npm run audit:failures
npx playwright install chromium
npm run preview -- --host 127.0.0.1 --port 5173

In another terminal, run npm run test:browser. The browser test uses the production build and saves actual screenshots and axe results. CHROMIUM_PATH may point to an installed Chromium binary.

Evidence boundaries

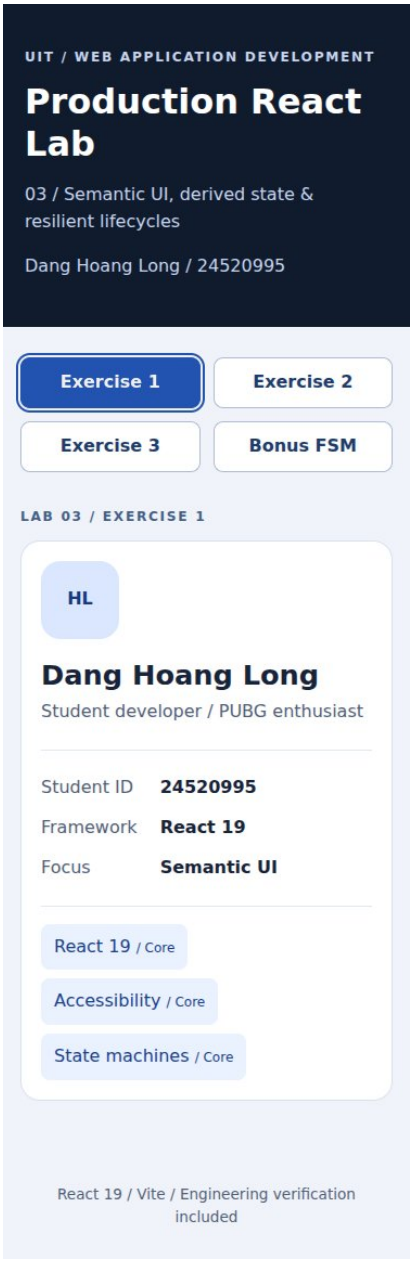
Lighthouse audits the first view only. Axe also scans the other views. Neither tool establishes complete WCAG conformance. Manual screen-reader review, a React DevTools Profiler recording and the instructor live defense remain student activities. Development StrictMode may invoke renders more than once; that must be distinguished from committed updates.

The avatar network request is deliberately blocked during automated browser tests to exercise fallback. The recorded console claim excludes that expected resource failure. The application contains no credentials, real telemetry subscription or claim of official affiliation.

06 / RESPONSIVE EVIDENCE

375px Mobile Layout

The browser test changes the viewport to 375 x 812 and measures document.scrollWidth against innerWidth in every exercise. Each assertion passed. These screenshots show natural full-page height, with wrapping navigation and controls.



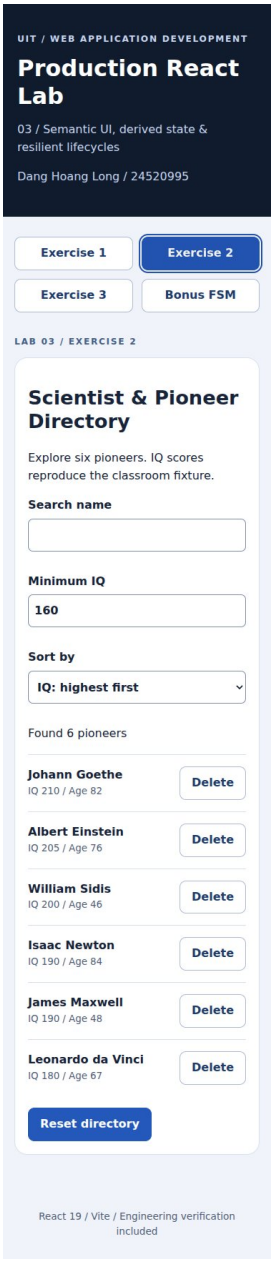


Figure 6. Exercise 1 and Exercise 2 at 375px. Source screenshots for Exercise 3 and the bonus are also included in evidence/.

Responsive behavior uses wrapping flex layouts, a single-column filter form below 600px and fluid card padding. Focus indicators remain visible. The skeleton animation respects prefers-reduced-motion.

Failure Analysis and Submission Notes

Faulty candidate	Reproduction and repair
In-place sorting	sort changes the original row ID order. The repaired toSorted selector preserves the source.
Uncanceled telemetry	A slower CES result overwrites a newer GE result. Aborting the old request retains GE.
Leaked ghosting timeout	A pending timeout overwrites COMMITTED with exit. AbortSignal cleanup preserves commitment.

These are isolated AI-style faulty candidates generated for controlled verification. They are explicitly labelled and excluded from production. The audit does not invent a prior DevTools discovery history. tests/failure-audit.mjs executes both faulty and repaired variants; AI_FAILURE_AUDIT.md documents reproduction, fixes and source files.

Task and Git structure

TASK_DECOMPOSITION.md maps ten focused commits to setup, semantic components, responsive styles, data normalization, search, deletion/reset, async telemetry, recovery UI, bonus FSM and final verification/report. Existing Lab_01 and Lab_02 content remains outside this addition.

Live defense preparation

Review MANUAL_ACCEPTANCE.md. Be ready to explain derived state, stable keys, effect dependencies and cleanup. Practice the instructor mutations: a watchlist filter, polling with pause and cleanup, and lifting selected state to App. Close AI assistants during the instructor examination.

References

Course source: Lab 03: Production React 19, KRAFTON PUBG VN & Situationship State Machine, 35 slides. MSc. Tran Vinh Khiem, UIT - VNUHCM. Exercise contracts: slides 13-23 and 31-32. Evaluation: slides 24-25 and 34.

React documentation: react.dev/reference/react/useEffect - synchronization, dependencies and cleanup.

React documentation: react.dev/reference/react/useId - accessible unique IDs.

Vite documentation: vite.dev/guide/ - development and production build workflow.